
Respostas Técnicas para o Artigo Científico

Aplicação de Redes Neurais Convolucionais 3D no Reconhecimento de Ações Humanas em
Vídeo:
Uma Proposta para Gamificação de Academias

Cassio Figueira Santos
Davi Castro Jorge da Costa
Jessica Rodrigues Martins
Larissa Miuki Kitayama
Paulo Gabriel Fernandes Ribeiro
Rafael Alvarenga Pereira Palomino

Orientador: Prof. Marcus Artiaga Colantoni
Uberlândia · 2026

Importante: Este trabalho é caracterizado no projeto de pesquisa como uma **investigação teórica com proposta conceitual** (Design Science Research). O protótipo implementado usa **MediaPipe Pose + classificador baseado em regras angulares em JavaScript**, não uma CNN 3D treinada. As seções abaixo descrevem com precisão o que foi e o que não foi realizado.

1. Qual dataset foi usado?

Situação real: Nenhum dataset de vídeos foi coletado ou utilizado para treinamento. O protótipo opera exclusivamente com a câmera em tempo real. A classificação é feita por regras angulares hardcoded no código — não há aprendizado de máquina supervisionado.

Nome do dataset	Não se aplica. Nenhum dataset foi coletado. O protótipo utiliza o feed da câmera em tempo real como entrada direta, sem etapa de treinamento sobre vídeos pré-gravados.
Link / origem	Não se aplica. O modelo de detecção de pose utilizado é o MediaPipe Pose , carregado via CDN: <code>cdn.jsdelivr.net/npm/@mediapipe/pose</code> (conforme <code>pose.js</code> , linha 58 do código-fonte).
Quantidade total de vídeos	Nenhum vídeo foi coletado. Não aplicável.
Quantidade de classes	5 classes , definidas diretamente no código-fonte (<code>classifier.js</code> , linha 8).
Nome das classes / exercícios	Conforme <code>classifier.js</code> linha 8: 1. Agachamento 2. Polichinelô 3. Flexão 4. Avanço 5. Joelho Alto
Quantidade de vídeos por classe	Não se aplica — não há dataset de vídeos.
Tipo do dataset	Não aplicável. Não foi usado dataset. A proposta teórica do artigo descreve o uso futuro de datasets públicos de reconhecimento de ações humanas para treinar uma CNN 3D.
Divisão treino / validação / teste	Não se aplica. Não há treinamento ML. A classificação é baseada em limiares angulares definidos manualmente no código.

2. Como os vídeos foram pré-processados?

Frames por vídeo	O sistema não processa vídeos gravados. Opera em tempo real, frame a frame, via câmera web. Um buffer de 5 frames consecutivos é mantido para estabilizar a detecção de exercício antes de confirmar uma troca (<code>classifier.js</code> , linha 45: <code>_bufferSize = 5</code>).
Resolução dos frames	640 × 480 pixels — configurado em <code>pose.js</code> , linhas 103-104: <code>width: 640, height: 480</code> .
Redimensionamento	O MediaPipe Pose realiza redimensionamento interno dos frames antes da inferência. Não há redimensionamento manual no código do protótipo.

Normalização dos pixels	O MediaPipe normaliza automaticamente as coordenadas dos landmarks para o intervalo [0, 1] relativo às dimensões do frame. Não há normalização adicional implementada no código.
Extração / amostragem de frames	Todos os frames são processados em tempo real, sem subamostragem. O MediaPipe Camera envia cada frame capturado para o modelo (pose.js, função start(), linha 93-99).
Formato final da entrada da rede	Frame RGB (640×480) → MediaPipe Pose → 33 landmarks 3D com coordenadas normalizadas (x, y, z) e score de visibilidade → cálculo de 11 ângulos articulares : leftKnee, rightKnee, leftHip, rightHip, leftElbow, rightElbow, leftShoulder, rightShoulder, trunkAngle, leftAnkle, rightAnkle (pose.js, função getKeyAngles, linhas 257-276).

3. Qual arquitetura de IA foi usada?

Distinção importante: O artigo propõe teoricamente o uso de CNN 3D. O protótipo implementado utiliza MediaPipe Pose (modelo pré-treinado do Google) combinado com um classificador baseado em regras angulares — **não é uma CNN 3D treinada pela equipe**.

Arquitetura implementada no protótipo

Tipo	MediaPipe Pose (BlazePose) + Classificador baseado em regras angulares. Abordagem híbrida: modelo pré-treinado para detecção de pose + lógica simbólica para classificação de exercícios.
Modelo de detecção de pose	MediaPipe Pose, carregado via CDN (cdn.jsdelivr.net/npm/@mediapipe/pose). Configurações usadas (pose.js, linhas 62-66): modelComplexity: 1 smoothLandmarks: true enableSegmentation: false minDetectionConfidence: 0.5 minTrackingConfidence: 0.5
Landmarks extraídos	33 landmarks corporais por frame, com coordenadas normalizadas (x, y, z) e score de visibilidade — padrão do MediaPipe Pose.
Classificador de exercícios	Baseado em regras angulares — não possui camadas treináveis. Componentes (classifier.js): • Cálculo de 11 ângulos articulares (pose.js, getKeyAngles) • Máquina de estados por exercício (up/down, open/closed etc.) • Buffer de estabilidade: 5 frames consecutivos • Sistema de pontuação por repetição e qualidade
Camadas / estrutura	Não aplicável ao classificador (baseado em regras, sem camadas neurais). O MediaPipe Pose possui arquitetura interna própria do Google, não configurável pelo protótipo.
Função de ativação	Não aplicável ao classificador do protótipo (limiares angulares, sem neurônios).

Função de saída	Exercício detectado dentre 5 classes, contagem de repetições, score de qualidade (0 a 1) e pontuação gamificada.
Número de parâmetros treináveis	Zero — o classificador é baseado em regras. O MediaPipe Pose é um modelo pré-treinado pelo Google, cujos parâmetros não são modificados.

Arquitetura proposta no artigo (não implementada nesta fase — trabalho futuro)

Tipo proposto	CNN 3D para reconhecimento de ações em vídeo, conforme discutido na revisão da literatura do projeto de pesquisa (Yang et al., 2024 — arquitetura STA-C3DL; Bang e Park, 2024).
Referência teórica	YANG, Fan et al. Action recognition in rehabilitation: combining 3D convolution and LSTM with spatiotemporal attention. <i>Frontiers in Physiology</i> , v. 15, p. 1472380, 2024.
Status	Proposta conceitual — não implementada. A implementação de CNN 3D é identificada no cronograma do projeto como etapa futura (fase Protótipo: 13/05 a 31/05/2026).

4. Como foi feito o treinamento?

Não houve treinamento de modelo de machine learning. O protótipo utiliza o MediaPipe Pose (pré-treinado pelo Google) para detecção de pose e um classificador baseado em regras angulares escritas manualmente em JavaScript. Não há etapa de treinamento, otimizador, função de perda ou épocas.

Linguagem	JavaScript (ES6+) — aplicação web client-side, sem backend de processamento.
Bibliotecas / frameworks	Conforme os arquivos do projeto: • @mediapipe/pose — detecção de landmarks corporais (pose.js) • @mediapipe/camera_utils — acesso e controle da câmera (pose.js) • Firebase Firestore — persistência de treinos e exercícios (config.js, app.js) • Firebase Auth — autenticação do usuário (config.js) • HTML5 Canvas API — renderização do esqueleto em tempo real (pose.js)
Número de épocas	Não se aplica — não houve treinamento de modelo.
Taxa de aprendizado	Não se aplica — não houve treinamento de modelo.
Otimizador	Não se aplica — não houve treinamento de modelo.
Função de perda	Não se aplica — não houve treinamento de modelo.
Tempo de treinamento	Não se aplica — não houve treinamento de modelo.
Ambiente / máquina	Desenvolvimento em máquina local com VSCode e navegador Chrome. A aplicação roda inteiramente no navegador do usuário (client-side), sem servidor de processamento.

5. Quais foram os resultados?

Situação real: Nenhuma métrica formal foi computada (acurácia, precisão, recall, F1-score, matriz de confusão). Como o sistema é baseado em regras e não houve treinamento ML, não existem curvas de loss/accuracy. O artigo obtém métricas de forma indireta, pela análise comparativa dos estudos citados (conforme seção 6.1 da Metodologia do projeto de pesquisa).

Acurácia no treino	Não se aplica — não houve treinamento de modelo.
Acurácia na validação	Não mensurada formalmente. O sistema foi testado manualmente pelos autores para verificar o funcionamento da detecção de cada exercício.
Acurácia no teste	Não mensurada formalmente.
Precisão	Não mensurada formalmente.
Recall / Revocação	Não mensurado formalmente.
F1-score	Não mensurado formalmente. O projeto de pesquisa utiliza F1-score de forma indireta , comparando resultados reportados nos artigos citados (ex.: Yang et al., 2024; Bang e Park, 2024) — não do protótipo próprio.
Matriz de confusão	Não gerada — não houve avaliação formal com dataset rotulado.
Gráficos de loss e accuracy	Não existem — não houve treinamento por gradiente descendente.
Classes com melhor desempenho	Com base nos testes manuais realizados pelos autores: o Agachamento e o Polichinelo apresentaram detecção mais consistente, por serem movimentos bilaterais com padrões angulares bem definidos.
Classes com maior dificuldade	Com base nos testes manuais: Avanço e Joelho Alto apresentaram maior dificuldade, pois ambos envolvem levantamento unilateral de perna, gerando confusão nas regras de detecção (classifier.js, linhas 231-247).

O que foi validado no protótipo

Funcionalidade	Status verificado
Detecção de Agachamento em tempo real	Funcionando — máquina de estados up/down (classifier.js, linha 288)
Detecção de Polichinelo em tempo real	Funcionando — estado open/closed por posição de braços e pés (linha 382)
Detecção de Flexão em tempo real	Funcionando — detecta posição horizontal + flexão de cotovelo (linha 414)
Detecção de Avanço em tempo real	Funcionando — diferença angular entre joelhos (linha 483)
Detecção de Joelho Alto em tempo real	Funcionando — joelho acima do quadril (linha 528)
Contagem de repetições	Funcionando para todas as 5 classes

Score de qualidade por repetição (0–1)	Funcionando — verifica profundidade, alinhamento, simetria
Sistema de pontuação gamificada	Funcionando — pontos base + bônus de qualidade por exercício
Feedback textual em tempo real	Funcionando — mensagens de correção e confirmação
Persistência de treinos (Firebase)	Implementado em app.js via Firestore
Registro de exercício personalizado	Implementado — captura de template angular via câmera

6. Quais foram as dificuldades e limitações?

Limitação	Descrição
Ausência de dataset e métricas formais	Por ser uma pesquisa teórica com protótipo conceitual, não foi coletado dataset nem mensuradas métricas de ML. Não é possível afirmar acurácia com precisão científica.
Classificador baseado em regras (não ML)	O classificador usa limiares angulares fixos (hardcoded), o que limita a generalização para diferentes biotipos, velocidades de execução e alturas de câmera.
Variação de ângulo de câmera	A detecção é otimizada para câmera frontal. A Flexão (push-up) é mais precisa com câmera lateral — conflito com o posicionamento padrão do protótipo.
Dependência de iluminação	O MediaPipe Pose reduz a confiança dos landmarks em ambientes com pouca luz ou contraluz, impactando diretamente a classificação.
Confusão entre Avanço e Joelho Alto	Ambos os exercícios envolvem levantamento unilateral de perna, gerando sobreposição nos critérios angulares. Identificado nos testes manuais dos autores.
Limitação biomecânica	O sistema avalia ângulos 2D (projeção da câmera). Erros tridimensionais como rotação de quadril e pronação de pé não são detectados.
Custo computacional no navegador	O processamento frame a frame do MediaPipe em JavaScript pode ser lento em dispositivos de baixo desempenho, especialmente mobile.
CNN 3D não implementada	A arquitetura proposta no artigo (CNN 3D) não foi implementada nesta fase. O sistema conceitual completo está previsto como trabalho futuro.

7. O que ainda falta fazer?

Ponto	Descrição
Coletar dataset formal de vídeos	Gravar vídeos com diferentes usuários, biotipos, ângulos de câmera e condições de iluminação para possibilitar treinamento e avaliação de um modelo de CNN 3D.
Implementar e treinar CNN 3D	Substituir o classificador de regras por uma CNN 3D treinada, conforme proposto no artigo. Referência: Yang et al. (2024) — STA-C3DL (3D CNN + LSTM + atenção espaço-temporal); Bang e Park (2024) — CNN em ensemble para exercícios.

Mensurar métricas formais	Após treinar o modelo, calcular acurácia, precisão, recall, F1-score e gerar matriz de confusão com dataset de teste separado.
Melhorar interface gamificada	Aprimorar o sistema de pontuação com ranking, conquistas, missões diárias e dashboard de evolução do usuário ao longo dos treinos.
Validar com usuários reais	Realizar testes de usabilidade e eficácia com praticantes de academia reais, não apenas com os autores do projeto.
Validar com profissional de Ed. Física	Submeter o sistema e os feedbacks gerados à avaliação de um especialista em biomecânica e treinamento físico para atestar correção técnica.
Otimizar para dispositivos móveis	Testar e otimizar o desempenho para smartphones, que são o principal dispositivo de acesso em contexto de academia.
Expandir número de exercícios	O código já suporta exercícios personalizados via gravação de template angular (ExerciseRecorder — classifier.js, linha 678). Ampliar o conjunto de exercícios nativos reconhecidos pelo sistema.